

DOCKER DEVOPS HANDS-ON LAB

End-to-End Practical Guide for macOS and Windows

Application: Flask Web Service + Redis

Duration: approximately 60-75 minutes of hands-on work

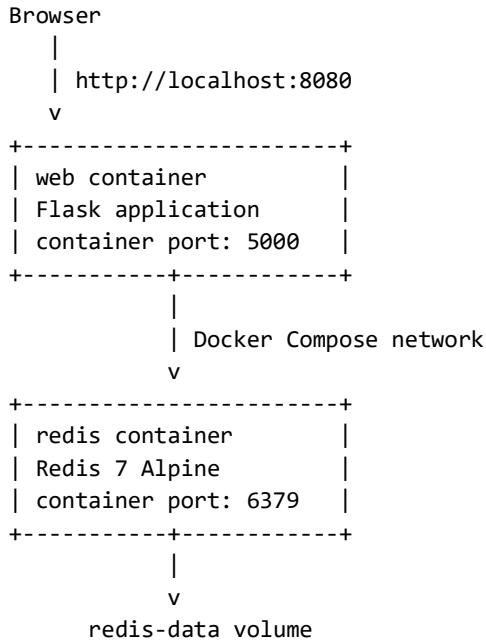
Lab goal: Build a Docker image, run and inspect containers, publish a port, work with the container filesystem, create persistent storage, understand registry operations, and deploy a two-service application using Docker Compose.

Prepared for classroom use | Version checked against Docker documentation in September 2026

1. What You Will Build

You will work with a small application that is deliberately simple enough for a classroom but realistic enough to demonstrate a multi-container deployment. The web service is a Flask application. Redis stores a page-view counter. Docker Compose runs both services together.

1.1 Final architecture



1.2 What each part demonstrates

Item	Why you are using it
Dockerfile	Build an application image from instructions.
Image	Immutable template used to create containers.
Container	Running instance of an image.
Port publishing	Expose container port 5000 through host port 8080.
docker exec / logs	Inspect and debug a running container.
Volume	Keep Redis data outside the disposable container layer.
Registry	Tag and optionally push an image for distribution.
Compose	Define and run multiple services as one application stack.

Important: Do not skip the checkpoints. The lab is written so that each checkpoint confirms the previous step before you continue.

2. Prerequisites and Installation

Use Docker Desktop and Linux containers. Docker Desktop includes Docker Engine, the Docker CLI, and Docker Compose, so you do not need to install Compose separately on macOS or Windows.

2.1 macOS

Docker Desktop supports supported/current macOS releases and requires at least 4 GB RAM. On Apple-silicon Macs, Rosetta 2 is no longer strictly required, although Docker documents it as recommended for some optional AMD64/Darwin tooling.

Official installation guide: [Docker Desktop for Mac](#)

2.1.1. Download Docker Desktop

Open the official Mac installation page and download the installer matching your Mac: Apple silicon or Intel.

2.1.2. Install

Open Docker.dmg and drag Docker to the Applications folder.

2.1.3. Start Docker Desktop

Open Applications -> Docker. Wait until Docker Desktop reports that Docker is running.

2.1.4. Verify the CLI

```
docker --version
docker compose version
```

Expected: Both commands should print version information. The exact version numbers can change over time; do not treat a specific version number in this guide as a requirement.

2.1.5. Run the standard test container

```
docker run hello-world
```

Checkpoint PASS: You should see Docker confirm that your installation is working. If this fails, stop here and use the troubleshooting section before continuing.

2.2 Windows

The recommended classroom setup is Docker Desktop with the WSL 2 backend and Linux containers. Docker documents WSL 2 as the default/recommended path for most Windows users. Docker Desktop can expose Docker commands from Windows Terminal/PowerShell, and WSL integration can be enabled for a Linux distribution.

Official Windows installation guide: [Docker Desktop for Windows](#)

Official WSL 2 integration guide: [Docker Desktop WSL 2 backend](#)

2.2.1. Check WSL

```
wsl --version
```

If WSL is not installed or the command does not show a current WSL version, open PowerShell as Administrator and run:

```
wsl --install
```

Restart Windows if prompted. After restart, open PowerShell again and run:

```
wsl --version  
wsl --update
```

Important: Do not install Docker Engine separately inside your WSL distribution for this lab. Use Docker Desktop as the Docker engine and enable WSL integration from Docker Desktop.

2.2.2. Download Docker Desktop

Download Docker Desktop for Windows from the official Docker page. During setup, use WSL 2 when offered.

2.2.3. Start Docker Desktop

Launch Docker Desktop from the Start menu. In Settings -> General, confirm the WSL 2 based engine is enabled when that option is available. In Settings -> Resources -> WSL Integration, enable your Ubuntu/other WSL distribution when you plan to use it.

2.2.4. Confirm Linux containers

This lab uses Linux containers. If Docker Desktop is in Windows-container mode, switch to Linux containers from the Docker menu.

2.2.5. Verify the CLI

```
docker --version  
docker compose version  
docker run hello-world
```

Checkpoint PASS: The hello-world container must complete successfully before starting the project.

3. Get the Lab Files Ready

Use the companion folder supplied with this guide: docker-devops-lab. You do not need Python, Flask, or Redis installed directly on the computer. Docker will provide the application runtime and Redis container.

3.1 Folder structure

```
docker-devops-lab/  
├─ app/
```

```

├── app.py
├── .dockerignore
├── Dockerfile
├── compose.yaml
├── requirements.txt
└── README.md

```

3.2 macOS setup

Extract the supplied ZIP file. Open Terminal and move into the extracted folder. For example:

```

cd ~/Downloads/docker-devops-lab
pwd
ls

```

You may store the project somewhere else. The key point is that the terminal must be inside the docker-devops-lab folder before you run docker build or docker compose commands.

3.3 Windows setup

Extract the supplied ZIP file into a convenient location such as Downloads or a course folder. Open PowerShell or Windows Terminal and move into the project folder. Example:

```

cd "$HOME\Downloads\docker-devops-lab"
Get-Location
Get-ChildItem

```

For Windows users who prefer WSL, copy or extract the project inside your WSL home directory and use the same Docker commands from the Linux terminal.

Common Windows mistake: File extensions can be hidden in File Explorer. Make sure the file is exactly named Dockerfile, not Dockerfile.txt, and compose.yaml is exactly compose.yaml.

4. Practical 1 - Inspect the Project Before Building

The aim is to understand what Docker will build before you ask Docker to build it.

4.1. Check the Dockerfile

```
cat Dockerfile
```

Windows PowerShell equivalent:

```
Get-Content Dockerfile
```

4.2. Understand the Dockerfile

- FROM python:3.12-slim -> starts from a small Python runtime image.
- WORKDIR /app -> sets the working directory inside the image.
- COPY requirements.txt . -> copies dependency information into the image.
- RUN pip install ... -> installs Flask and Redis Python libraries.

- COPY app/ ./app/ -> copies the application source into the image.
- EXPOSE 5000 -> documents the application port used by the container.
- CMD [...] -> starts the Flask application when the container runs.

4.3. Check Compose

```
cat compose.yaml
```

Windows PowerShell:

```
Get-Content compose.yaml
```

5. Practical 2 - Build a Docker Image

This is the first major practical objective: turn the source code and Dockerfile into an image.

5.1. Build the image

```
docker build -t docker-devops-lab:v1 .
```

What the final dot means: The dot is the build context. It tells Docker to use the current project folder as the context from which files referenced by COPY can be read.

5.2. Confirm the image exists

```
docker images
```

5.3. Inspect the image metadata

```
docker image inspect docker-devops-lab:v1
```

Expected: You should see docker-devops-lab with tag v1 in docker images. The image size may vary slightly because base images and dependencies change over time.

6. Practical 3 - Run a Container and Publish a Port

The image is not the running application. The next step creates a container from the image.

6.1. Run in foreground mode

```
docker run --name docker-lab-web-fg -p 8081:5000 docker-devops-lab:v1
```

Open a browser and visit: <http://localhost:8081>

Why 8081:5000?: 8081 is the port on your computer. 5000 is the port inside the container where Flask

listens. The format is HOST_PORT:CONTAINER_PORT.

6.2. Stop the foreground container

Return to the terminal running the container and press Ctrl+C.

6.3. Confirm the stopped container exists

```
docker ps
docker ps -a
```

Checkpoint PASS: docker ps should not show docker-lab-web-fg as running. docker ps -a should show it with an Exited status.

7. Practical 4 - Detached Mode, Logs, and Lifecycle

7.1. Remove the previous stopped container

```
docker rm docker-lab-web-fg
```

7.2. Run in detached mode

```
docker run -d --name docker-lab-web -p 8080:5000 docker-devops-lab:v1
```

7.3. Confirm it is running

```
docker ps
```

7.4. Open the application

Visit <http://localhost:8080> in a browser. The application homepage should load.

7.5. Read container logs

```
docker logs docker-lab-web
```

7.6. Follow logs live

```
docker logs -f docker-lab-web
```

Press Ctrl+C to stop following the logs. This does not stop the container; it only ends log-follow mode.

7.7. Stop and start the container

```
docker stop docker-lab-web
docker ps -a
docker start docker-lab-web
```

```
docker ps
```

7.8. Restart the container

```
docker restart docker-lab-web
```

8. Practical 5 - Inspect the Container Filesystem

8.1. Open a shell inside the running container

```
docker exec -it docker-lab-web sh
```

8.2. Inspect the application directory

```
pwd
ls
ls /app
cat /app/app/app.py | head -n 20
```

8.3. Exit the container shell

```
exit
```

Windows note: Run Docker commands from PowerShell, Windows Terminal, or a WSL terminal. The command syntax above is the Docker syntax and remains the same across macOS and Windows.

9. Practical 6 - Demonstrate the Disposable Container Layer

This experiment shows why important application data should not be stored only in a container writable layer.

9.1. Create a temporary file inside the container

```
docker exec docker-lab-web sh -c "echo temporary-data > /tmp/demo.txt"
```

9.2. Confirm the file exists

```
docker exec docker-lab-web sh -c "cat /tmp/demo.txt"
```

9.3. Remove the container

```
docker rm -f docker-lab-web
```

9.4. Create a new container from the same image

```
docker run -d --name docker-lab-web -p 8080:5000 docker-devops-lab:v1
```

9.5. Check whether the temporary file survived


```
docker exec docker-lab-web sh -c "cat /tmp/demo.txt"
```

Expected: The file should not exist. This illustrates the disposable nature of a container filesystem layer. The exact error text is normal and is not a Docker failure.

10. Practical 7 - Named Volume and Persistence

Now we store data in a Docker-managed named volume so the data can survive container removal.

10.1. Stop and remove the current standalone web container

```
docker rm -f docker-lab-web
```

10.2. Create a named volume

```
docker volume create docker-lab-data
```

10.3. Check the volume

```
docker volume ls
```

10.4. Run a Redis container using the volume

```
docker run -d --name docker-lab-redis-volume -v docker-lab-data:/data redis:7-alpine redis-server --appendonly yes
```

10.5. Write a value into Redis

```
docker exec docker-lab-redis-volume redis-cli SET student_message "Docker volume works"
docker exec docker-lab-redis-volume redis-cli GET student_message
```

10.6. Remove the Redis container but keep the volume

```
docker rm -f docker-lab-redis-volume
```

10.7. Create another Redis container using the same volume

```
docker run -d --name docker-lab-redis-volume-2 -v docker-lab-data:/data redis:7-alpine redis-server --appendonly yes
```

10.8. Verify the data survived

```
docker exec docker-lab-redis-volume-2 redis-cli GET student_message
```

Expected: The command should return Docker volume works. The container was deleted; the named volume remained.

11. Practical 8 - Image Tagging and Registry

A registry stores and distributes images. Docker Hub is one example. Cloud registries include Amazon ECR, Azure Container Registry, Google Artifact Registry, and GitHub Container Registry.

11.1. Tag the image for Docker Hub

Replace YOUR_DOCKERHUB_USERNAME with your own Docker Hub username. Do not use angle brackets in the command.

```
docker tag docker-devops-lab:v1 YOUR_DOCKERHUB_USERNAME/docker-devops-lab:v1
```

11.2. List the tagged image

```
docker images
```

11.3. Optional: log in to Docker Hub

```
docker login
```

Follow the prompt. Do not paste credentials into the lab document or classroom chat.

11.4. Optional: push the image

```
docker push YOUR_DOCKERHUB_USERNAME/docker-devops-lab:v1
```

11.5. Optional: pull the image on another machine

```
docker pull YOUR_DOCKERHUB_USERNAME/docker-devops-lab:v1
```

Classroom recommendation: If Docker Hub authentication is blocked by campus policy, treat tagging and push/pull as a demonstration rather than a required checkpoint. The core learning outcome is image creation and execution.

12. Practical 9 - Deploy the Application with Docker Compose

This is the main advanced practical. Compose describes the web service and Redis service together, connects them on a private Docker network, and attaches a named volume to Redis.

12.1. Inspect compose.yaml

```
cat compose.yaml
```

Windows PowerShell:

```
Get-Content compose.yaml
```

12.2. Validate the Compose file

```
docker compose config
```

Expected: Docker should print the normalized Compose configuration. If YAML indentation is wrong, the command will show a parsing error. Fix indentation before continuing.

12.3. Build and start the full application

```
docker compose up -d --build
```

12.4. Check service status

```
docker compose ps
```

12.5. Open the web application

Visit <http://localhost:8080>

12.6. Check the application health endpoint

Visit <http://localhost:8080/health>

```
curl http://localhost:8080/health
```

If curl is unavailable in a particular terminal, use the browser instead.

12.7. Test Redis-backed page views

Open <http://localhost:8080/counter> several times. Each response should increase the `page_views` number because Redis is storing the counter.

12.8. Inspect Compose logs

```
docker compose logs
docker compose logs web
docker compose logs redis
```

12.9. Follow web logs

```
docker compose logs -f web
```

Refresh the browser a few times, observe traffic in the logs, then press Ctrl+C.

12.10. See the running containers

```
docker ps
```

12.11. Stop the stack

```
docker compose down
```

12.12. Start it again

```
docker compose up -d
```

12.13. Verify persistence

Open /counter again. The counter should continue from the previous stored value because the Redis data is stored in the named redis-data volume.

Checkpoint PASS: At this point you have built an image, published a port, run a foreground and detached container, inspected the filesystem, demonstrated disposable storage, used a volume, understood registry tagging, and deployed a multi-container application with Compose.

13. What compose.yaml Means

Compose element	Meaning
services.web	The Flask application service.
build: .	Build the web image from the Dockerfile in the current directory.
ports	Maps host port 8080 to container port 5000.
environment	Tells Flask how to find Redis by service name.
depends_on	Starts Redis as a dependency of the web service; it does not by itself guarantee application-level readiness.
services.redis	Runs Redis from the official image.
volumes	Mounts redis-data into Redis /data for persistence.

14. Docker Swarm - Theory / Optional Demonstration

Swarm is included in the syllabus, but building a multi-node cluster is intentionally not required for this short lab. Understand the model: Compose is suited to defining multi-container applications, while Swarm mode provides cluster management and service orchestration across Docker hosts.

```

Single host
|
+-- Docker Compose

Multiple Docker hosts
|
+-- Swarm manager
|   |
|   +-- services / replicas
|
+-- Swarm workers
  
```

Important vocabulary: manager node, worker node, service, replica, desired state. A full Swarm deployment normally requires multiple hosts/network connectivity and is therefore outside the core 60-75 minute student practical.

14.1. Optional single-node demonstration

```
docker swarm init
```

```
docker service create --name swarm-web --publish 8085:80 nginx
```

```
docker service ls
```

```
docker service ps swarm-web
```

```
docker service rm swarm-web
```

```
docker swarm leave --force
```

Warning: Run the Swarm demonstration only if your instructor explicitly asks for it. It changes the Docker engine into Swarm mode and is not required for the core lab deliverable.

15. Final Cleanup

Perform cleanup so future practicals start from a known state.

15.1. Stop and remove the Compose application

```
docker compose down
```

15.2. List containers

```
docker ps -a
```

15.3. Remove leftover lab containers, if any

```
docker rm -f docker-lab-redis-volume-2 2>/dev/null || true
```

Windows PowerShell alternative if the previous command is not accepted:

```
docker rm -f docker-lab-redis-volume-2
```

15.4. List volumes

```
docker volume ls
```

15.5. Optional: remove lab volumes

```
docker volume rm docker-lab-data
```

Do not remove redis-data unless you are finished with the Compose persistence demonstration.

16. Troubleshooting - Common Student Errors

Symptom	What to do
docker: command not found / not recognized	Docker Desktop is not installed, not started, or its CLI is not available on PATH. Start Docker Desktop,

Symptom	What to do
	wait for it to finish starting, then open a new terminal. On Windows, verify Docker Desktop is using the WSL 2 backend.
Cannot connect to the Docker daemon	Docker Desktop is not running or the engine has not finished starting. Open Docker Desktop and wait until it reports running. Re-run docker info.
docker compose: command not found	Use Docker Desktop rather than an old standalone Compose installation. Check docker compose version. Docker Desktop includes Compose.
WSL is not installed	Open Administrator PowerShell and run wsl --install, restart if requested, then verify with wsl --version. See the official Microsoft/Docker setup instructions.
Docker Desktop says Windows containers	Switch Docker Desktop to Linux containers. This lab uses Linux images such as python:3.12-slim and redis:7-alpine.
port is already allocated / address already in use	Another application is using port 8080/8081. Run docker ps to see existing containers, stop the conflicting lab container, or temporarily change the host port, for example -p 8090:5000. When using Compose, change only the host-side port in compose.yaml.
Docker build fails on COPY	Make sure you are running docker build from the docker-devops-lab directory and that Dockerfile, requirements.txt, and app/app.py exist with the exact names.
Dockerfile.txt on Windows	Windows may hide extensions. Turn on View -> Show -> File name extensions in File Explorer and rename Dockerfile.txt to Dockerfile.
YAML parse error in compose.yaml	Check indentation. YAML uses spaces, not tabs. Compare your file with the supplied compose.yaml. Run docker compose config before docker compose up.
/health returns 503 or Redis unavailable	Check docker compose ps. Then run docker compose logs redis and docker compose logs web. Make sure the Compose stack is running and the environment variable REDIS_HOST is redis, not localhost.
web container exits immediately	Run docker logs docker-lab-web. Most classroom cases are caused by a build error, a missing source file, or a command copied incorrectly. Rebuild with docker compose up -d --build.
Browser shows connection refused	First check docker ps. Then check the host port in the docker run or compose.yaml command. Confirm Docker is running and try http://localhost:8080 explicitly.
Permission denied on macOS/Windows project files	Keep the project in a normal user-owned folder. On

Symptom	What to do
	Windows with WSL, keeping the project inside the Linux filesystem can avoid slow or awkward cross-filesystem behavior.
docker exec says container is not running	Start the container first with <code>docker start <container-name></code> , then run <code>docker exec</code> again.
Data disappeared after container removal	That is expected for data stored only in the container writable layer. Use a named volume for persistent data.

17. Diagnostic Command Set

When something fails, use these commands before changing random settings. They tell you what Docker is currently doing.

```

docker info
docker version
docker ps
docker ps -a
docker images
docker volume ls
docker network ls
docker logs <container-name>
docker inspect <container-name>
docker compose ps
docker compose logs

```

18. Student Check-off

- ☐ Docker Desktop installed and running.
- ☐ `docker --version` works.
- ☐ `docker compose version` works.
- ☐ `docker run hello-world` works.
- ☐ `docker-devops-lab:v1` image built successfully.
- ☐ Standalone container ran on `http://localhost:8080` or `8081`.
- ☐ Foreground vs detached mode demonstrated.
- ☐ `docker logs` and `docker exec` used.
- ☐ Temporary container filesystem behavior observed.
- ☐ Named volume created and persistence verified.
- ☐ Registry tag created; push/pull understood or demonstrated.
- ☐ `docker compose up -d --build` completed successfully.
- ☐ `/health` returned a healthy/connected response.
- ☐ `/counter` increased across requests.
- ☐ Compose logs inspected.
- ☐ `docker compose down` completed.

19. Quick Viva / Discussion Questions

Question	Expected answer
What is the difference between an image and a container?	An image is the packaged template; a container is a runtime instance created from an image.
What does -p 8080:5000 mean?	Host port 8080 is published to container port 5000.
Why did the temporary file disappear?	It was stored in the container writable layer, which is removed with the container.
Why does Redis use a volume?	To persist Redis data outside the disposable container layer.
Why can the web service use redis as the hostname?	Compose creates a network and provides service-name based DNS for services in the same application.
What is the role of a registry?	To store and distribute container images.
Why use Compose?	To define and operate multiple related services as a single application stack.
When would Swarm be relevant?	When Docker services need cluster-level scheduling and orchestration across multiple Docker hosts.

20. Official References

- [Install Docker Desktop on Mac](#)
- [Install Docker Desktop on Windows](#)
- [Docker Desktop WSL 2 backend](#)
- [Docker Compose installation overview](#)
- [Docker volumes](#)
- [Docker Swarm mode](#)

Note: Docker Desktop requirements, UI labels, licensing terms, and exact version numbers can change. The installation instructions above are written to minimize version-specific assumptions and are based on the official Docker documentation checked in September 2026.